

Q1. What is LangChain?

LangChain is a framework used to build applications powered by Large Language Models (LLMs). It helps developers connect LLMs with tools, databases, APIs, memory, and external data sources to create intelligent applications like chatbots, AI agents, and RAG systems.

Q2. Why do we use LangChain?

We use LangChain because working directly with LLM APIs can become complex in real-world projects. LangChain provides reusable components for prompts, chains, memory, retrieval, tools, and agents, making development faster and more organized.

Q3. What problems does LangChain solve?

LangChain solves problems like:

- Managing prompts
 - Connecting LLMs with external data
 - Building multi-step AI workflows
 - Creating AI agents
 - Handling memory and conversation context
-

Q4. What are some common use cases of LangChain?

Common use cases include:

- Chatbots
 - Question-answering systems
 - RAG applications
 - AI agents
 - Document summarization
 - Content generation
 - Code assistants
-

Q5. What is the difference between using only an LLM API and using LangChain?

Using only an LLM API gives access to the model, but LangChain provides a complete framework around it. LangChain helps manage prompts, chains, retrieval, tools, and workflows more efficiently.

2. LangChain Components

Q1. What are the main components of LangChain?

Main components include:

- Models
 - Prompts
 - Output Parsers
 - Chains
 - Runnable
 - Document Loaders
 - Text Splitters
 - Vector Stores
 - Retrievers
 - Tools
 - Agents
 - Memory
-

Q2. Why is LangChain modular?

LangChain is modular so developers can use only the components they need. For example, a simple chatbot may only need prompts and models, while a RAG system may also need retrievers and vector stores.

Q3. Which LangChain components are mostly used in RAG applications?

In RAG applications, commonly used components are:

1. Document Loaders
2. Text Splitters

3. Embedding Models
 4. Vector Stores
 5. Retrievers
 6. LLMs
 7. Chains or Runnables
-

3. LangChain Models

Q1. What are models in LangChain?

Models are components that interact with AI models like OpenAI GPT, Gemini, Claude, or local models. LangChain mainly supports:

- LLMs
 - Chat Models
 - Embedding Models
-

Q2. What is the difference between an LLM and a Chat Model?

LLM	Chat Model
Works with plain text input	Works with message-based input
Older style interaction	Modern conversational interaction
Example: text completion	Example: GPT chat models

Q3. What are embedding models?

Embedding models convert text into numerical vectors. These vectors help in similarity search, semantic search, and retrieval systems.

Q4. Why are embeddings important in RAG?

Embeddings help find similar documents based on meaning instead of exact keywords. This improves document retrieval accuracy in RAG systems.

4. Prompts in LangChain

Q1. What is a prompt in LangChain?

A prompt is the instruction or input given to the LLM. LangChain helps structure prompts dynamically using templates.

Q2. What is PromptTemplate?

PromptTemplate allows developers to create reusable prompts with variables. It makes prompts cleaner and easier to manage.

Q3. What are the benefits of using prompt templates?

Benefits include:

- Reusability
 - Dynamic input handling
 - Cleaner code
 - Better prompt management
-

Q4. What is the difference between static and dynamic prompts?

Static Prompt	Dynamic Prompt
Fixed text	Uses variables
Cannot change easily	Flexible and reusable

5. Output Parsers in LangChain

Q1. What is an output parser?

Output parsers structure the raw output from the LLM into a specific format like JSON, lists, or objects.

Q2. Why are output parsers useful?

They help make AI responses predictable and easier to use in applications.

Q3. What are common types of output parsers?

Common parsers include:

- String Parser
- JSON Parser
- Structured Output Parser
- Pydantic Parser

Q4. What happens if we do not use output parsers?

Without parsers, LLM responses may be inconsistent and difficult to process programmatically.

6. Chains in LangChain**Q1. What is a chain in LangChain?**

A chain connects multiple components together so the output of one step becomes the input of another.

Q2. Why are chains useful?

Chains help automate multi-step workflows like:

1. User Input
2. Prompt Creation
3. LLM Response
4. Output Processing

Q3. What is the difference between Sequential Chain and Parallel Chain?**Sequential Chain**

Tasks run one after another

Slower

Parallel Chain

Tasks run simultaneously

Faster

Sequential Chain

Parallel Chain

Used when steps depend on each other Used when tasks are independent

Q4. When should we use parallel chains?

Parallel chains should be used when multiple independent tasks can run at the same time, such as generating a summary and quiz simultaneously.

7. Runnables in LangChain

Q1. What are runnables in LangChain?

Runnables are the new execution system in LangChain used to create flexible and composable AI workflows.

Q2. Why were runnables introduced?

Runnables provide:

- Better flexibility
 - Easier composition
 - Streaming support
 - Parallel execution
 - Cleaner architecture
-

Q3. What is the difference between old chains and runnables?

Old Chains

Runnables

Less flexible

More flexible

Older architecture

Modern architecture

Limited composition

Easy composition

Q4. What are common runnable methods?

Common methods include:

- `invoke()`
 - `batch()`
 - `stream()`
-

8. Document Loaders

Q1. What is a document loader?

Document loaders are used to load data from different sources like PDFs, text files, websites, or databases.

Q2. Why are document loaders important?

They help bring external knowledge into AI applications.

Q3. What types of sources can document loaders handle?

They can load:

- PDFs
 - DOCX files
 - Websites
 - CSV files
 - Databases
 - APIs
-

9. Text Splitters

Q1. What is a text splitter?

A text splitter breaks large documents into smaller chunks.

Q2. Why is text splitting important?

LLMs have token limits, so large documents must be divided into manageable chunks.

Q3. What is chunk size?

Chunk size defines how much text each chunk contains.

Q4. What is chunk overlap?

Chunk overlap repeats some text between chunks to maintain context continuity.

10. Vector Stores

Q1. What is a vector store?

A vector store is a database that stores embeddings for similarity search.

Q2. Why are vector stores used?

They help retrieve semantically similar documents quickly.

Q3. Name some popular vector databases.

Popular vector databases include:

- Pinecone
 - Chroma
 - Weaviate
 - FAISS
-

11. Retrievers in LangChain

Q1. What is a retriever?

A retriever fetches the most relevant documents from a vector store based on a query.

Q2. What is the difference between a retriever and a vector store?

Vector Store

Retriever

Stores embeddings Fetches relevant documents

Vector Store	Retriever
Database layer	Search layer

Q3. Why are retrievers important in RAG?

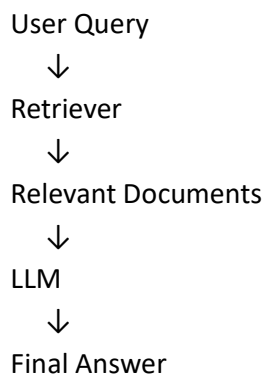
Retrievers help provide relevant context to the LLM before generating an answer.

12. RAG in LangChain

Q1. What is RAG?

RAG stands for Retrieval-Augmented Generation. It combines document retrieval with LLM generation.

Q2. How does RAG work?



Q3. What are the advantages of RAG?

Advantages include:

- Reduces hallucinations
 - Uses external knowledge
 - Gives more accurate answers
 - Supports private company data
-

Q4. What is the role of embeddings in RAG?

Embeddings help retrieve semantically relevant documents from the vector database.

13. Tools in LangChain

Q1. What are tools in LangChain?

Tools are external functions or services that AI models can use to perform actions.

Q2. Why do agents need tools?

Agents need tools because LLMs alone cannot access real-time data or external systems.

Q3. Give examples of tools.

Examples:

- Web Search
 - Calculator
 - Database Query
 - Weather API
 - File Reader
-

14. Tool Calling

Q1. What is tool calling?

Tool calling is the process where an LLM decides when and how to use a tool.

Q2. Why is tool calling important?

It allows AI systems to interact with external systems and perform real-world tasks.

Q3. What happens in the tool-calling workflow?

User Request



LLM decides tool



Tool executes



Result returned to LLM



Final Response

15. AI Agents

Q1. What is an AI agent?

An AI agent is a system where the LLM can think, decide actions, use tools, and generate responses autonomously.

Q2. What is the difference between chains and agents?

Chains

Fixed workflow

Predefined steps

Predictable flow

Agents

Dynamic decision-making

Chooses actions dynamically

Flexible behavior

Q3. How do AI agents work?

User Input



Reasoning



Choose Tool



Execute Tool



Observe Result



Generate Final Answer

Q4. What are the advantages of AI agents?

Advantages include:

- Dynamic decision-making
 - Tool usage
 - Multi-step reasoning
 - Automation of complex tasks
-

Q5. What are the limitations of AI agents?

Limitations include:

- Higher cost
- Slower execution
- Possible wrong decisions
- Complex debugging